

QUICK REFERENCE GUIDE

RAG

LangChain

Cheatsheet

RAG



Stop Googling the same snippets.
Every pattern you need — in one place.

Data Loading

Chunking

Embeddings

Vector Stores

Retrieval

RAG Pipeline

Data Loading

PURPOSE Load documents from various file formats so they can be processed by your pipeline.

COMMON LOADERS

- **PyPDFLoader** — for PDFs
- **TextLoader** — for plain text files
- **CSVLoader** — for CSV data

```
from langchain_community.document_loaders import PyPDFLoader
```

```
loader = PyPDFLoader("file.pdf")  
documents = loader.load()
```

Document Chunking

PURPOSE Split documents into smaller, manageable chunks to improve retrieval accuracy.

✗ Too large → messy, unfocused answers

✓ Just right → precise retrieval

COMMON SPLITTER RecursiveCharacterTextSplitter

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)
chunks = splitter.split_documents(documents)
```

PURPOSE Convert text chunks into numerical vector representations that can be stored and searched.

POPULAR EMBEDDINGS

- **OpenAIEmbeddings** — fast, production-ready
- **HuggingFaceEmbeddings** — open source, free

```
from langchain_openai import OpenAIEmbeddings

embeddings = OpenAIEmbeddings()
```



Embeddings are the bridge between human language and vector math. Choose your model based on cost vs. quality tradeoff.

Vector Stores

PURPOSE Store and retrieve vector embeddings efficiently for similarity search.

Chroma

Local, lightweight, great for dev

FAISS

Fast, open source, Meta-built

Pinecone

Managed cloud, production scale

Weaviate

Open source + cloud hybrid

```
from langchain_community.vectorstores import FAISS

vectorstore = FAISS.from_documents(
    chunks, embeddings
)
```

PURPOSE Convert a vector store into a retriever and fetch the most relevant document chunks based on user queries.

```
retriever = vectorstore.as_retriever(  
    search_type="similarity", k=3  
)  
  
docs = retriever.invoke("What is LangChain?")
```



k=3 means return top 3 most similar chunks



Higher k = more context, but slower and noisier

PURPOSE Combine retrieval with a language model to answer questions based on retrieved context — Retrieval-Augmented Generation.

MODERN PATTERN (LCEL) `create_retrieval_chain`

```
from langchain.chains import create_retrieval_chain
from langchain.chains.combine_documents import create_stuff_documents_chain
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template(
    "Use the context to answer: {context} Question: {input}"
)
combine_docs_chain = create_stuff_documents_chain(llm, prompt)
rag_chain = create_retrieval_chain(retriever, combine_docs_chain)

result = rag_chain.invoke({"input": "Explain RAG"})
```

 LCEL pattern: compose retriever + `stuff_documents_chain` into a single `rag_chain`. Supports streaming, batching, and tracing out of the box.



Follow for more

AI 🤖 × Cloud ☁ × Capital Markets 📊

Raghavendra Bagalkoti

Capital Markets Technology

💬 You've read the cheatsheet. Now the real question: are you still building RAG from scratch — or are you letting LangChain do the heavy lifting? Drop your stack below 📌